Procesadores de lenguajes

Santiago Javier Proaño Yépez

<2026-10-09 vie>

Contents

1 Procesador de lenguaje **1**
- 1.1 Traductor . . . 2
- 1.2 Compilador . . . 2
- 1.3 Ensamblador . . . 2
- 1.4 Intérprete . . . 2
- 1.5 Simulador . . . 2

2 El compilador puede coexistir con el intérprete ATTACH **2**

3 Caso C ATTACH **3**
- 3.1 Preprocesador ATTACH . . . 3
- 3.2 Linker ATTACH . . . 4
- 3.3 Loader . . . 4

4 Definiciones **5**
- 4.1 Transliterador . . . 5
- 4.2 Parser . . . 5
- 4.3 Máquna de estados . . . 5

1 Procesador de lenguaje

Básicamente ingresa un lenguaje o programa y realiza una operación sobre ellas. Existiendo 5 tipos de procesadores de lenguaje:

1.1 Traductor

Entra un algo y lo cambia algo equivalente. Mantiene el significado relevante de la entrada, solo cambia su representación. Por ejemplo el traductor de Google cambia de un idioma a otro sin cambiar su significado.

1.2 Compilador

Es una especie de traductor, cambia el programa fuente a un código objeto para ser ejecutado: Conceptos generales es el más rápido para ser ejecutado.

1.3 Ensamblador

Otro tipo de traductor, pero lo que traduce es lenguaje ensamblador, y lo pasa al lenguaje de máquina, 1's y 0's. Las instrucciones ensamblador corresponden a las ISA's (instruccion set arquitecture) es decir vienen dichas por el set de instrucciones del procesador.

1.4 Intérprete

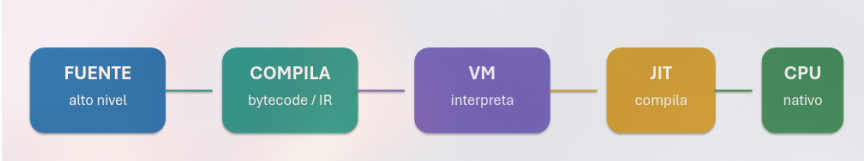
También traduce las instrucciones del código fuente, pero lo hace linea por linea y encima las ejecuta linea por linea también, el problema es que los intérpretes antiguos cuando existía un error tenia que volver a interpretar desde la linea 1 otra vez. Esto se solucionó con los interpretes incrementales que no tienen que interpretar todo de nuevo. Ejemplo python.

1.5 Simulador

Lo que hace es imitar el comportamiento de otra máquina a través de software, mantiene variables de la maquina simulada y repite el Circulo básico del CPU.

2 El compilador puede coexistir con el intérprete ATTACH

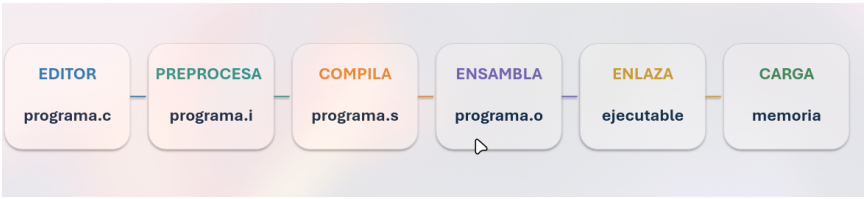
En la actualidad los sistemas modernos combinan estas dos estrategias para mejorar la ejecución. Por ejemplo Pasan del programa fuente y se compila hasta obtener la representación intermedia luego este es interpretado y vuelto a compilar para ser ingresado al procesador.



Pese a que no se debería clasificar la tecnología con una sola etiqueta, muchas veces se puede elegir una sobre otra dependiendo el objetivo.

- Si se necesita tener un desempeño nativo es mejor usar la compilación.
- Si se necesita tener interacción inmediata con el programa es mejor usar un interprete.
- Si se esta usando lenguaje de bajo nivel es mejor usar un ensamblador.
- Y por ultimo si se necesita probar otra arquitectura es mejor usar un simulador.

3 Caso C ATTACH



Este es el pipeline de como funciona un compilador. Hay nuevas etapas nuevas que no se mencionaron en la compilación pero estas son a groso modo:

3.1 Preprocesador ATTACH

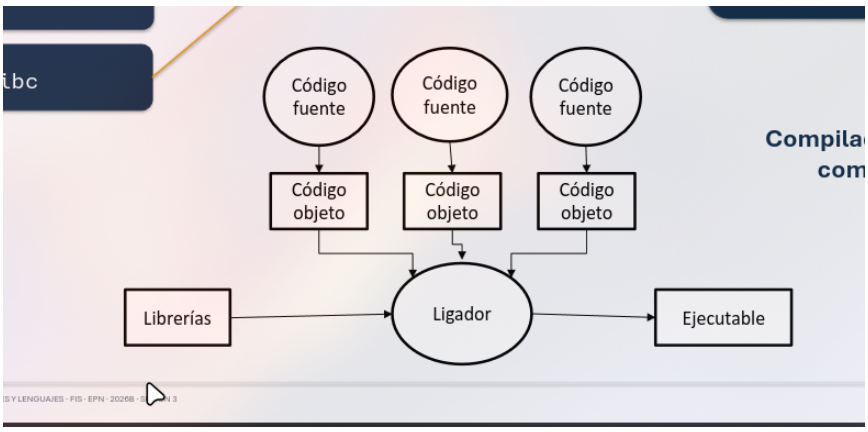
Básicamnte junta todo el programa para hacer la compilación, puede incluso hacer transformaciones. Por ejemplo si se tiene en el codigo un include `uno.c` al momento de compilar se junta eeste en el codigo pricipal, vease la imagen



.

3.2 Linker ATTACH

Integra todas las referencias de los módulos de un sistema y las librerías que se usan para crear el ejecutable.Por ejemplo, si yo tengo en un archivo .c una llamada de suma que esta en math.c al compilar y tener el .o el linker pone la definicion de dicha llamada. Ojo el compilador no es el que hace esta tarea, sino linker.



3.3 Loader

Es quien carga el ejecutable producido por el linker en la memoria del procesador y alli se empieza a ejecutar el programa.

Lo que hace es:

1. Leer el encabezado para saber el formato, el tamaño etc.
2. Luego crea el espacio de memoria donde coloca el código, los datos y reserva la pila y el heap. La pila es la memoria que se usa para las llamadas a funciones donde la ultima llamada que entra es la que sale primero. Por otro lado el heap se usa para la memoria dinámica, como una bodega para guardar los datos cuando no sabemos cuanta memoria va a ocupar.
3. Si es dinámico el enlace, se cargan las bibliotecas y se resuelven las dependencias
4. Ajustar direcciones si es necesario: Se pone al programa en una dirección por seguridad)

5. Pone el contador de programa en el punto de entrada, alli el CPU empieza el fetch y sigue con Circulo básico del CPU

4 Definiciones

4.1 Transliterador

El transliterador básicamente cambia de un símbolo a otro símbolo, por ejemplo cambiar los números romanos a numero decimales. La cosa es que cada símbolo cambia según una tabla, no cambia el significado ni analiza nada solo cambia los símbolos.

4.2 Parser

El programa recibe una secuencia de tokens y decide si forma una estructura permitida por una gramática. Si es correcta se acepta y se hace un árbol con esa estructura, básicamente forma parte de un analizador sintáctico Conceptos generales.

4.3 Máquna de estados

Básicamente entra una cadena de entradas y se pasan por estados, dependiendo la entrada puede cambiar a uno u otro estado. Un autómata básicamente